

Preparação Completa — Entrevista Buzzlabs

PARTE 1: ESTRATÉGIA

Objetivo da entrevista

Não é provar que você sabe Clojure. Eles já viram o currículo. O objetivo é fazer com que saiam pensando: *"esse cara pensa como a gente pensa, resolve o tipo de problema que a gente resolve, e vai comunicar bem com nossos clientes."*

Sua vantagem real

Você tem algo raro: 7 anos de Clojure em produção **em consultoria**, não em empresa de produto. Isso significa:

- Você adaptou a mesma stack para domínios completamente diferentes
- Você aprendeu a defender decisões técnicas para clientes não-técnicos
- Você entregou sistemas completos (schema → UI) com autonomia

Os três medos do entrevistador

Todo entrevistador está inconscientemente verificando:

1. **Vai entregar?** — histórico de projetos completos, métricas reais
2. **Vai se comunicar?** — clareza técnica, honestidade sobre gaps
3. **Vai se integrar?** — colaboração, autonomia calibrada, sem ego

Tom correto

- **Confiante, não arrogante.** Você tem 7 anos de experiência real.
- **Honesto sobre gaps, não defensivo.** Kafka — você sabe o que não tem e por quê.
- **Curioso, não ansioso.** Faça perguntas genuínas.

PARTE 2: O QUE O HUMANO VAI PENSAR

Ao ver "Projeto Independente" no topo (2025-Atual): → A carta de apresentação já endereça isso. Mencione o Ark Engine como trabalho deliberado e delimitado — não como "o que fiz enquanto procurava emprego".

Ao ver XTDB: → Explicação em 30s: "É o Datomic com bitemporalidade nativa — mesmo modelo Datalog, criado pela Juxt. A Nubank usa Datomic; XTDB resolve o mesmo problema com uma dimensão a mais de tempo."

Ao ver "Consultoria em Software" sem nome de empresa: → Se perguntarem: "Prefiro não divulgar o nome do cliente por razões de confidencialidade — é o padrão em consultoria."

Ao ver Go no currículo de um candidato Clojure: → Posicione como um **benchmark deliberado**. "Quis validar até onde o modelo mental funcional de imutabilidade e snapshots (que Clojure facilita) poderia ser levado em termos de latência bruta ($\sim 43\mu s$) e economia de infraestrutura."

Ao ver Octopus Pay (2022-2025) com 7 bullets densos: → Esteja preparado para detalhar qualquer bullet. Destaque Pathom 3, Clara Rules e Fulcro RAD.

Ao ver "Bacharelado em Física" sem conclusão: → "Migrei para Engenharia de Software em 2024, que é mais alinhado com o que pratico."

Ao ver Polyolith nas habilidades: → Diferencial positivo: "Esse cara pensa em arquitetura de verdade."

Sobre a Indicação (Leandro): → Trunfo de credibilidade. Mencione que ele recomendou fortemente a cultura da Buzzlabs.

PARTE 3: NARRATIVA PESSOAL (90 SEGUNDOS)

"Sou engenheiro Clojure/ClojureScript fullstack há 7 anos, todos em consultoria. Cada projeto foi um domínio diferente — gateway de pagamentos, marketplace B2B, motor multi-adquirente — e em todos entregava as duas pontas: do schema Datomic ao componente na tela. De julho de 2025 até agora trabalhei no Ark Engine — infraestrutura de trading algorítmico. Comecei com Clojure/XTDB/Polyolith e evolui para Go para benchmarks de latência extrema, preservando os mesmos princípios funcionais. Agora quero voltar para consultoria, onde produzo melhor em equipe."

PARTE 4: RESPOSTAS TÉCNICAS EM CAMADAS

Clojure / Ecossistema

Experiência: 7 anos em produção. Pathom 3, Datomic, Clara Rules, Fulcro RAD. **Insights:** Sistemas que duram têm dados como centro, não objetos. Pathom (resolvers), Datomic (schema como dado), Clara (regras como dado).

Pathom 3: Motor de resolução de grafos. Elimina o acoplamento 'de onde vem o dado'. O cliente pede via EQL e o motor descobre o caminho automaticamente. Útil para integrar 25+ gateways (Octopus Pay) de forma canônica.

Datomic: Base Datalog imutável. Escolho quando auditabilidade e evolução de schema são requisitos. *as-of queries* eliminam a necessidade de tabelas de auditoria manuais.

Clara Rules: Motor Rete. Evita 'if-else que virou árvore'. Regras declarativas e testáveis para cálculos complexos (comissão, frete, impostos).

Fulcro RAD: Produtividade via atributos declarativos. Gera forms, reports e routing a partir do schema de domínio.

Polyolith: Arquitetura de monorepo com componentes caixa-preta. Elimina acoplamento implícito. Ex: Risk Guard isolado no Ark Engine.

XTDB: Datalog imutável com bitemporalidade nativa (Valid Time vs Transaction Time). Essencial para eliminar *lookahead bias* em trading.

Kafka / Mensageria

Gap: Lacuna operacional, não conceitual. Padrões (offsets, consumer groups, exactly-once) implementados via Redis Streams e NATS JetStream. Interesse direto em fechar essa lacuna com Kafka.

PARTE 5: PERGUNTAS COMPORTAMENTAIS (STAR)

- **Problema Difícil:** Migração Pathom 2 → 3 em produção sem downtime, mapeando grafo em camadas.
- **Discordância:** Defender Datomic traduzindo técnica para "risco de negócio" (auditoria financeira).
- **Cliente Errado:** Separar o resultado desejado da solução técnica proposta e oferecer alternativa viável.
- **Falha/Atraso:** Integração VTEX (comportamentos não documentados) e como o fix incluiu normalização de erros e testes reais.

PARTE 6: PERGUNTAS DIFÍCEIS

Ark Engine lucra? Infra sólida, falta *edge* de estratégia (próximos passos: funding rate arbitrage). **Por que saiu?** Foco integral no desenvolvimento do Ark Engine. **Trabalho em equipe?** 7 anos de consultoria provam adaptabilidade e colaboração. **Gaps?** Kafka em produção, times maiores, novos domínios.

PARTE 7: PERGUNTAS PARA O ENTREVISTADOR

- Qual projeto mais desafiador hoje?
- Ciclo típico de projeto (proposta → entrega)?
- Onboarding de novos devs?
- Espaço para Open Source?

PARTE 8: CONCEITOS PARA REVISAR

- Resolver Pathom, Query Datalog básica.
- Atom + swap!, Malli schemas.
- map, reduce, transduce (transducers).
- core.async (chan, go, park vs block).

PARTE 9: ARMADILHAS E FECHAMENTO

- Não fale mal do cliente anterior.
- Não minimize o Go (foi boa engenharia).
- Não demonstre desespero.
- **Declare interesse no fim:** "Qual o próximo passo do processo?"